

DAY-4 — Linux Permissions

CyberNotes • Detailed Beginner Reference (Improved Edition)

Learning goal: Build a practical, working understanding of **linux permissions**, connect the concept to real cybersecurity work, and practice it safely in a local lab.

Study rule: Understand a command or concept before copying it. Security tools are powerful because they interact with real systems — your practice environment should always be intentionally controlled and authorized.

1. Core Concepts

Core idea: Linux Permissions is a building block for practical cybersecurity. This section explains what each term actually means, where you'll encounter it, and how it connects to the bigger picture.

Why it matters: Security work depends on understanding normal system and network behavior before you can recognize abnormal behavior. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting — the threads running through this whole 20-day series.

Vocabulary covered: users, groups, rwx, chmod, chown, umask, special permissions, permission troubleshooting.

Reference	What to remember
<code>ls -l</code>	view permission bits
<code>chmod 640 file</code>	set mode
<code>chown user file</code>	change owner
<code>chgrp group file</code>	change group
<code>umask</code>	view default permission mask
<code>id</code>	show user/group IDs

2. Concept Explanations

Each term below is explained in plain language, with a note on why it matters for security work. Read these before running any commands.

1. users

Every file and process on Linux belongs to a user, identified internally by a numeric UID. The special user ``root`` (UID 0) can bypass most permission checks, which is exactly why attackers who compromise a low-privilege account often try to "escalate" to root. ``id`` shows your current user and UID.

2. groups

A group is a collection of users that share access rights, letting you grant a permission to many people at once instead of one file at a time. A user can belong to multiple groups simultaneously, and group membership is a common way least-privilege access is enforced (see Day 6).

3. rwx

Permissions are expressed as read (r), write (w), and execute (x), applied separately to the file's owner, its group, and everyone else — visible as the ten-character string at the start of `ls -l` output (e.g. `-rwxr-xr--`). Read lets you view content, write lets you modify it, and execute lets you run it as a program or, for a directory, enter it.

4. chmod

`chmod` changes the rwx permission bits on a file, either symbolically (`chmod u+x file`) or numerically (`chmod 750 file`, where 7=rwx, 5=r-x, 0=---). Over-permissive settings (like world-writable config files) are a classic, easily-avoidable security weakness that `chmod` exists to fix.

5. chown

`chown` changes which user owns a file (and optionally its group in one command, e.g. `chown user:group file`). Incorrect ownership is a common cause of "permission denied" errors and can also be a red flag during an investigation if a file's owner doesn't match who should have created it.

6. umask

`umask` sets the default permission mask applied to newly created files and directories, effectively defining a baseline of "how open is too open" for your system. A tighter umask (e.g. 027) means new files start more locked-down by default, reducing accidental exposure.

7. special permissions

Beyond rwx, Linux has special bits: SUID (run as the file's owner, not the invoking user), SGID (run as the file's group, or inherit group on new files in a directory), and the sticky bit (only the owner can delete files in a shared directory, like `/tmp`). SUID binaries are a frequent target in privilege-escalation research because a misconfigured one can let a normal user gain root.

8. permission troubleshooting

When something fails with "permission denied," the systematic approach is: check `ls -l` on the file, check `id` for your own user/groups, and check whether the issue is the file's permissions, its directory's permissions, or ownership. Changing multiple things at once (chmod and chown together, for instance) makes it harder to tell which change actually fixed — or broke — the problem.

3. Safe Practical Lab

Step 1. Define the lab target and the exact question you are trying to answer.

Step 2. Run the smallest safe observation or command that can answer that question.

Step 3. Read the output carefully; record important fields, errors, timestamps, and assumptions.

Step 4. Change one variable at a time and compare the result.

Step 5. Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
touch permissions.txt
ls -l permissions.txt
chmod 600 permissions.txt
ls -l permissions.txt
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool's output with proof of a vulnerability or compromise — output is evidence, not a conclusion.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it hard to tell what actually fixed or broke the lab.
- Saving real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target just because it's reachable — reachability is never the same as authorization.

Troubleshooting checklist:

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Answer these in your own words after finishing the lab — they're designed to check real understanding, not memorization.

Q1. What problem does users solve, in your own words?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first, and why?

Notes: _____

Q4. What could cause a misleading result or false positive here?

Notes: _____

Q5. How would you document your observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain linux permissions in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit written permission.

Keep a written scope: target, allowed actions, time window, and stop conditions. Never scan or test random public systems.

When practicing, save your notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-4, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands — it's to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-4 — Linux Permissions

Three things I learned:

1. _____
1. _____
1. _____

Commands / concepts to memorize:

- `ls -l` — view permission bits
- `chmod 640 file` — set mode
- `chown user file` — change owner
- `chgrp group file` — change group
- `umask` — view default permission mask
- `id` — show user/group IDs

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.